ESTUDO DE DUPLICAÇÕES EM SUBSISTEMAS DO KERNEL LINUX: ANÁLISE, REFATORAÇÃO E CONTRIBUIÇÕES

Guilherme Ivo Bozi, Gabriel Sousa Santos de Almeida,

Natália Salvino André, Paulo Meirelles

Instituto de Matemática, Estatística e Ciência da Computação/Universidade de São Paulo

{guilherme.bozi, gabrielsousa230}@usp.br

Objetivos

O kernel Linux é um dos mais relevantes projetos de software livre, apresentando crescimento contínuo em tamanho e número de contribuidores[6]. Nesse contexto, torna-se importante a aplicação de boas práticas de engenharia de software para garantir a qualidade e a manutenibilidade do código. Portanto, este trabalho tem como objetivo identificar e mitigar duplicações de código no kernel Linux com o apoio da `ArKanjo`[2], ferramenta de detecção de duplicação de código, bem como a análise e seleção de casos para aplicação de técnicas de refatoração, priorizando critérios de manutenibilidade[1]. A partir dessa análise, pretende-se propor e submeter *patches* (contribuições) com refatorações para a comunidade do kernel Linux. Além disso, busca-se documentar o processo e as interações com a comunidade por meio de observação participante[5, 4], sistematizando as lições aprendidas e refletindo sobre os aspectos socio-técnicos do processo de contribuição.

Métodos e Procedimentos

O `ArKanjo` possui uma arquitetura de dois estágios (Figura 1), atualizada neste trabalho em relação ao modelo inicial[2]. O *Preprocessor* processa a base de código e gera representações das unidades analisadas, enquanto o *Query Responder* permite realizar consultas sem reprocessar o código. A ferramenta passou a utilizar o `Tree-sitter`, ampliando o suporte a diferentes linguagens e possibilitando análises baseadas em ASTs. Também foi incorporado um novo método de similaridade baseado em *embeddings* de código, além de melhorias na extensibilidade dos comandos e no suporte a saídas estruturadas e customizáveis.

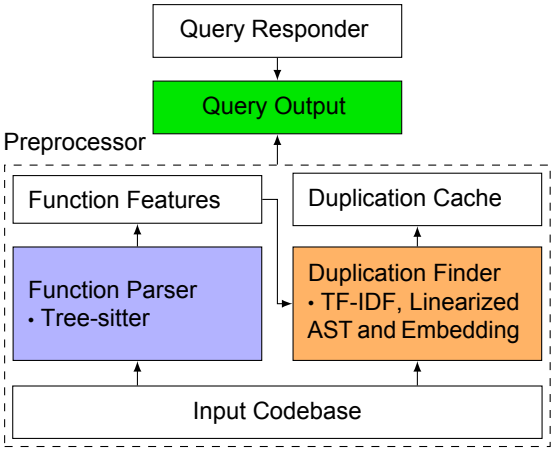


Figura 1: Diagrama de arquitetura com os componentes da ferramenta `ArKanjo`.

Para avaliar a aplicabilidade do `ArKanjo` no desenvolvimento do kernel Linux, foi conduzido um estudo etnográfico e multimétodo, dividido em duas fases, acompanhando o processo desde a

identificação de duplicações até a implementação e submissão de possíveis contribuições.

Na **Fase 1**, o `ArKanjo` foi utilizado para identificar duplicações com 90% e 100% de similaridade, selecionar os casos mais promissores e analisar seu potencial de refatoração. Na **Fase 2**, os *patches* foram implementados e submetidos aos mantenedores, incluindo revisões e ajustes quando solicitados. O processo foi aplicado aos subsistemas AMDGPU e IIO, em dois contextos: na **Observação Participante**, os autores realizaram as contribuições, buscando compreender as dificuldades do processo de desenvolvimento e documentar as interações com os mantenedores. No **Estudo com Participantes**, alunos da disciplina MAC0470/5856 (USP, 2026) atuaram como contribuidores, podendo realizar a Fase 1 ou utilizar duplicações previamente identificadas pelos autores e concentrar-se na Fase 2.

Resultados

Foram submetidos 17 *patches* aos subsistemas AMDGPU e IIO, dos quais 8 foram incorporados, 4 permanecem em revisão e 5 não foram incorporados, resultando em uma remoção líquida de 1.096 linhas de código. As interações com os mantenedores indicaram que a aplicação do princípio DRY (*Don't Repeat Yourself*)[3] no kernel é contextual, considerando aspectos como o isolamento entre gerações de hardware, a simplicidade e a legibilidade.

Tabela 1: Resumo das submissões de *patches* de refatoração, agrupadas por *status*.

Status	**Patches**	**Diff**
Incorporados	8	`+1541/-2637`
Em revisão	4	`+140/-248`
Não incorporado	5	`+388/-584`

Conclusões

Os resultados mostram que o processo de transformação de duplicações em contribuições vai além da identificação automática, evidenciando a importância da análise do contexto e da interação com a comunidade. Nesse sentido, o `ArKanjo` mostrou-se útil para apoiar a exploração de uma base de código extensa e a identificação de oportunidades de contribuição. Esses resultados evidenciam seu potencial como ferramenta de apoio à participação de novos contribuidores e à descoberta de oportunidades de melhoria no kernel Linux.

Os autores declaram não haver conflito de interesses. **Guilherme** contribuiu com o envio de 4 *patches* e na elaboração deste resumo. **Gabriel** contribuiu com o envio de 2 *patches* e na elaboração deste resumo. **Natália** colaborou com um *patch* e na revisão deste texto. **Paulo** orientou na concepção e planejamento do estudo. Todos os autores aprovaram a versão final deste resumo.

O presente trabalho foi realizado com apoio da Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP), Brasil. Processo nº 2025/22416-1.

Referências bibliográficas

[1] E. L. G. Alves, P. D. L. Machado, T. Massoni, and S. T. C. Santos. A Refactoring-Based Approach for Test Case Selection and Prioritization. In *Proc. 8th AST*, 2013.

[2] L. Arcanjo, D. Tadokoro, and P. Meirelles. ArKanjo: a tool for detecting function-level Code Duplication in the Linux Kernel. In *DebConf25*. IRISA, 2025.

[3] L. Arcanjo, D. Tadokoro, M. Spessoto, R. Passos, and P. Meirelles. When Do You Repeat Yourself? Voices from the Trenches of Linux Kernel Maintainers on Code Duplication. In *TechDebt'26*. ACM, 2026.

[4] C. Robson and K. McCartan. *Real World Research: A Resource for Users of Social Research Methods in Applied Settings*. John Wiley & Sons, 4 edition, 2016.

[5] C. B. Seaman. Qualitative Methods. In *Guide to Advanced Empirical Software Engineering*, pages 35–62. Springer, 2008.

[6] Willie. Linux kernel contributors and lines of code statistics 2026, 2026.